Lab 2: Arrays and ArrayLists

Algorithms and Data Structures (010153523) | Semester 1/2026

Duration: 2 hours (in-lab) | **Topic:** Fixed arrays, insertion/deletion, dynamic resizing, amortised analysis

Objectives

- Declare, initialise, and traverse one-dimensional arrays in Java.
- Implement insertion and deletion in a sorted array and understand the $O(n)$ shift cost.
- Build a `Scoreboard` class that maintains a sorted top-k array.
- Implement a generic `ArrayList<E>` backed by an `Object[]` that doubles when full.
- Analyse the amortised $O(1)$ cost of `add` using the accounting method.

Quick Reference – Arrays

Declaring arrays.

```
int[] a = new int[5];                        // all zeros
String[] names = {"Alice", "Bob", "Carol"};
int len = a.length;                          // field, NOT a method
```

Shifting right to insert at index k (loop *backward*):

```
for (int i = last; i >= k; i--) a[i+1] = a[i];
a[k] = newValue;
```

Complexity. Random access: $O(1)$. Insert/delete (keeping order): $O(n)$.

Quick Reference – ArrayList

Resizing strategy. When full, allocate double the capacity and copy all elements.

```
private void resize() {
    Object[] bigger = new Object[data.length * 2];
    for (int i = 0; i < size; i++) bigger[i] = data[i];
    data = bigger;
}
```

Amortised analysis. Starting from capacity 1, resizes at sizes $1, 2, 4, \ldots, n$ cost $1 + 2 + 4 + \cdots + n = 2n - 1$ total copies for n adds: $O(1)$ amortised per `add`.

Key operations.

`get(i)`/`set(i,e)`	$O(1)$	always
`add(e)` at tail	$O(1)$	amortised
`add(i,e)` at index	$O(n)$	shift cost
`remove(i)`	$O(n)$	shift cost

Quick Experiments (Try It)

Try It 1: Off-by-one index

```
int[] a = {10, 20, 30, 40, 50};
for (int i = 0; i <= a.length; i++)    // deliberate bug
    System.out.println(a[i]);
```

What exception is thrown and on which iteration? Fix the loop bound.

Try It 2: Watch capacity double

Add a `capacity()` accessor to your `ArrayList`. Start with capacity 2 and append elements one by one; print the capacity after each add. Count the total number of element copies across 16 appends. Is it less than 2×16?

In-Lab Exercises (target: 2 hours)

In-Lab Exercise 1: Array Statistics *(20 min)*

Read n integers from the user (first read n, then the values). Print: the minimum, maximum, and average (as `double`). Handle $n = 1$ correctly.

In-Lab Exercise 2: `Scoreboard` Class *(40 min)*

Implement `GameEntry` (name + score, `toString()`) and `Scoreboard(int capacity)` that keeps at most `capacity` entries in descending score order.

- `add(GameEntry e)`: insert if the board is not full or `e` beats the lowest score; shift as needed.
- `remove(int i)`: remove entry at index `i` and shift left.
- `toString()`: print all entries, one per line.

Test with at least 6 entries and one removal.

In-Lab Exercise 3: Implement `ArrayList<E>` *(50 min)*

Build the class with an initial capacity of 4:

- `int size()`, `boolean isEmpty()`, `int capacity()`.
- `E get(int i)` — throws `IndexOutOfBoundsException` if out of range.
- `void set(int i, E e)`.
- `void add(E e)` — appends; doubles capacity when full.
- `void add(int i, E e)` — inserts at index `i`, shifting right.
- `E remove(int i)` — removes and returns; shifts left.
- Shrink policy: if after removal `size` $<$ `capacity`/4, halve capacity (min 4).
- `String toString()` — e.g. `[A, B, C] (size=3, cap=4)`.

Test: append 6 elements, insert at index 2, remove index 0, print. Verify one resize occurred.

AI Collaboration

Suggested prompts:

- "Why must I loop *backwards* when shifting array elements right for insertion?"
- "Explain amortised $O(1)$ for `ArrayList.add` using the accounting/potential method."

Verify: Ask the AI to predict the total element copies for 32 appends starting from capacity 1 with a doubling strategy. Compute it yourself and compare.

Take-Home (Paper Work). Solve the following on paper without using a computer or IDE. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to trace and reason about code during the written exam.

Trace & Predict 1: Insertion shift

Trace the following code for `a = {2, 5, 8, 0}`, `size = 3`, `v = 6`. Show the array after each loop iteration and the final state.

```
int pos = size;
while (pos > 0 && a[pos-1] > v) { a[pos] = a[pos-1]; pos--; }
a[pos] = v;
```

Trace & Predict 2: ArrayList capacity trace

Starting from capacity 2, trace `capacity()` after each call (use shrink-at-$<$ 1/4 policy, min 4): `add(A)`, `add(B)`, `add(C)`, `add(D)`, `add(E)`, `remove(0)`, `remove(0)`, `remove(0)`.

Write Code on Paper 1: Binary search on a sorted array

Write `int binarySearch(int[] a, int n, int target)`: returns the index of `target` in the sorted array of length `n`, or -1 if absent. Use the iterative approach.

Draw on Paper 1: Amortised accounting table

Draw a table: add #, element, capacity before, resize? (Y/N), copies made. Fill it for adds 1–16 starting from capacity 1. Sum the copies and show the total is less than $2 \times 16 = 32$.

Submission. Save files as `lab2_ex*.java` and submit on the course site.